

|                                                                                                                                                                                                                                    |                                                                                                                                                    |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
|  <p><b>ESPE</b><br/>UNIVERSIDAD DE LAS FUERZAS ARMADAS<br/>INNOVACIÓN PARA LA EXCELENCIA</p> <p><b>TEMA:</b></p> <p><b>Patrones de Diseño</b></p> | Departamento: Departamento de Ciencias de la Computación                                                                                           |
|                                                                                                                                                                                                                                    | Carrera: Ingeniería de Software<br>Asignatura: Análisis y Diseño de Software<br>NRC: 30723                                                         |
|                                                                                                                                                                                                                                    | Integrantes Grupo 3: <ul style="list-style-type: none"> <li>• Ayuquina Danny.</li> <li>• Cañarte Saray.</li> <li>• Villagómez Doménica.</li> </ul> |

## PROYECTO: FICITAS

### 1. Patrones de Diseño

#### 1.1. Decorator

La elección del patrón Decorator para FiCitas responde a un principio fundamental de la ingeniería de software: el principio de responsabilidad única (SRP). Los servicios centrales del sistema —gestión de citas, pacientes y planificación— deben concentrarse exclusivamente en las operaciones de negocio y persistencia de datos. Incorporar validaciones y verificaciones de disponibilidad directamente en estas clases violaría este principio, generando componentes sobrecargados y difícilmente testeables de forma aislada.

El patrón Decorator resuelve este problema mediante composición en lugar de herencia. Al definir la interfaz IServicio como contrato común, se garantiza que cualquier combinación de decoradores sea transparente para el cliente: FormularioGestionCitas no necesita saber si está interactuando con el servicio base desnudo o con varias capas de lógica envueltas alrededor de él. Esta transparencia hace que el sistema sea abierto a la extensión y cerrado a la modificación (principio OCP), ya que agregar una nueva capa —por ejemplo, un decorador de autorización de roles— no requiere modificar ninguna clase existente.

Desde la perspectiva de los requisitos del proyecto, la separación es especialmente valiosa: ValidacionDecorator y DisponibilidadDecorator encapsulan reglas que pueden cambiar con frecuencia (formatos de cédula, políticas de horario, restricciones de consultorio) sin afectar la lógica de persistencia. Además, la cadena de decoradores puede configurarse dinámicamente en tiempo de ejecución, permitiendo que ciertos formularios apliquen solo validación básica sin necesidad de verificar disponibilidad, lo que aporta flexibilidad operativa sin duplicar código.

Finalmente, la separación de responsabilidades facilita las pruebas unitarias: cada decorador puede probarse inyectándole un mock del servicio subyacente, y los componentes concretos (ControlCita, etc.) pueden validarse de forma independiente sin que las reglas de negocio interfieran. Esto reduce el acoplamiento entre capas y mejora la mantenibilidad del sistema a largo plazo.

| Elemento del patrón Decorator | Clase en el proyecto                                                                 | Responsabilidad                                                                                                                                                                                       |
|-------------------------------|--------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Component (interfaz)          | IServicio                                                                            | Define el contrato base con el método ejecutar(datos: Map): Resultado, que todos los servicios concretos y decoradores deben implementar.                                                             |
| Concrete Component            | ControlCita / ControlPaciente / ControlPlanificacion                                 | Implementaciones base de las operaciones CRUD; delegan la persistencia de datos a sus respectivos repositorios sin incluir lógica de validación ni disponibilidad.                                    |
| Base Decorator                | ServicioBaseDecorator                                                                | Clase abstracta que implementa IServicio, contiene una referencia protegida al componente envuelto y delega las operaciones al servicio interno.                                                      |
| Concrete Decorator 1          | ValidacionDecorator                                                                  | Añade validación de campos obligatorios, formatos (cédula, teléfono, consistencia edad/fecha de nacimiento) y reglas de negocio antes de ejecutar las operaciones del servicio base. (REQ003, REQ004) |
| Concrete Decorator 2          | DisponibilidadDecorator                                                              | Verifica disponibilidad de horarios, consultorios y fisioterapeutas, detectando conflictos de asignación antes de confirmar la operación al servicio base. (REQ003, REQ006)                           |
| Client                        | FormularioGestionCitas / FormularioGestionPacientes / FormularioGestionPlanificacion | Construye la cadena de decoradores e interactúa únicamente con la interfaz IServicio, sin conocer las implementaciones concretas ni el número de capas aplicadas.                                     |
| Entidad                       | Paciente / Cita / Planificacion                                                      | Representan los datos del dominio que fluyen a través de la cadena de decoradores hacia el servicio base.                                                                                             |
| Resultado                     | Resultado                                                                            | Encapsula la respuesta de cada operación, incluyendo estado de éxito/error y mensaje asociado.                                                                                                        |
| Repositorio (interfaz/impl)   | RepositorioPaciente / RepositorioCita / RepositorioPlanificacion                     | Definen y ejecutan el contrato de persistencia; solo son accedidos por los componentes concretos, nunca por los decoradores.                                                                          |

## 2.2.Observer

El patrón Observer es la solución más adecuada cuando un sistema necesita propagar efectos secundarios ante eventos sin que los emisores de esos eventos conozcan ni dependan de quienes los procesan. En FiCitas, los controladores (ControlCita, ControlPaciente, etc.) son responsables de coordinar las operaciones del dominio, pero no debería ser su responsabilidad saber que existe un sistema de auditoría ni un sistema de notificaciones. Codificar esas dependencias directamente en los controladores los convertiría en componentes altamente acoplados, difíciles de mantener y de extender. Al introducir GestorEventos como sujeto observable central, se invierte esta dependencia: los controladores solo necesitan publicar un Evento tras cada operación relevante, y el

gestor se encarga de notificar a todos los observadores registrados. Esto respeta el principio de inversión de dependencias (DIP) y permite que el sistema de auditoría y el de notificaciones evolucionen de forma completamente independiente.

La clase LogAuditoriaObserver justifica plenamente este enfoque: la trazabilidad completa de todas las acciones del sistema —quién hizo qué, cuándo y sobre qué datos— es un requisito transversal que afecta a todos los módulos. Implementarla mediante Observer garantiza que ningún controlador pueda olvidarse de registrar una acción, porque el registro ocurre de forma automática y centralizada ante cualquier evento publicado. Esto hace que el mecanismo de auditoría sea robusto frente a errores de omisión y coherente en todo el sistema.

Por su parte, NotificacionObserver demuestra otro beneficio clave del patrón: la capacidad de añadir nuevas reacciones ante eventos existentes sin modificar los emisores. Si en el futuro se requiere enviar un resumen semanal o activar una integración con un sistema externo ante la creación de una cita, bastará con registrar un nuevo observador en GestorEventos, sin tocar ControlCita. La combinación con el patrón Strategy en los canales de notificación (ICanal, CanalWhatsApp, CanalEmail) refuerza aún más esta extensibilidad, permitiendo que el canal de entrega sea configurable sin afectar la lógica del observador. En conjunto, el uso del patrón Observer permite que FiCitas cumpla sus requisitos de auditoría y notificación de manera desacoplada, extensible y verificable, alineándose con los principios SOLID y con las buenas prácticas de arquitectura de software orientada a objetos.

| Elemento del patrón Observer | Clase en el proyecto | Responsabilidad                                                                                                                                                                         |
|------------------------------|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Subject / Observable         | GestorEventos        | Mantiene la lista de observadores registrados y los notifica ante cualquier evento del sistema mediante notificar(evento: Evento). Es el punto central de publicación de eventos.       |
| Observer (interfaz)          | IObservadorAuditoria | Define el contrato que todos los observadores deben cumplir: actualizar(evento: Evento): void. Permite que GestorEventos notifique sin conocer las implementaciones concretas.          |
| Concrete Observer 1          | LogAuditoriaObserver | Recibe los eventos publicados y persiste la auditoría completa en RepositorioAuditoria: usuario, rol, fecha, hora, acción realizada, datos anteriores y datos nuevos. (REQ000 – REQ006) |
| Concrete Observer 2          | NotificacionObserver | Recibe eventos relacionados con citas (creación, cancelación, reprogramación) y delega el envío de notificaciones a ControlNotificacion a través del canal configurado. (REQ002)        |
| Event / Value Object         | Evento               | Encapsula la información del evento ocurrido: tipo de acción, usuario responsable, rol, fecha, hora, entidad afectada y datos anteriores/nuevos.                                        |

|                                    |                                                                                                      |                                                                                                                                                                                                                       |
|------------------------------------|------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Publisher<br>(Concrete<br>Subject) | ControlAutenticacion /<br>ControlCita / ControlPaciente /<br>ControlPerfil /<br>ControlPlanificacion | Todos los controladores contienen una referencia a<br>GestorEventos y publican un Evento tras cada<br>operación relevante, sin conocer quién lo procesará.                                                            |
| Canal de<br>Notificación           | ICanal / CanalWhatsApp /<br>CanalEmail                                                               | Definen el contrato de envío (enviar(dest, msg):<br>boolean); las implementaciones concretas<br>determinan el canal usado por ControlNotificacion al<br>procesar la notificación del observer. (Strategy<br>auxiliar) |
| Repositorio de<br>Auditoría        | RepositorioAuditoria                                                                                 | Persiste los registros de auditoría generados por<br>LogAuditoriaObserver; provee consulta por filtros<br>para trazabilidad completa.                                                                                 |
| Repositorio de<br>Notificación     | RepositorioNotificacion                                                                              | Almacena el historial de notificaciones enviadas con<br>su estado de entrega (Enviado / Pendiente / Fallido)<br>para seguimiento.                                                                                     |